

Logging using SLF4J

Exercise 1: Logging Error Messages and Warning Levels

Task: Write a Java application that demonstrates logging error messages and warning levels using SLF4J.

Step-by-Step Solution:

1. Add SLF4J and Logback dependencies to your `pom.xml` file:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version> </dependency>
```

2. Create a Java class that uses SLF4J for logging:

```
import org.slf4j.Logger; import
org.slf4j.LoggerFactory;
```

```
public class LoggingExample {   private static final Logger logger =
LoggerFactory.getLogger(LoggingExample.class);
```

```
    public static void main(String[] args) {
logger.error("This is an error message");
logger.warn("This is a warning message");
    }
}
```

```
[INFO] Running com.example.MyServiceArgumentTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceArgumentTest
[INFO] Running com.example.MyServiceMultipleReturnsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceMultipleReturnsTest
[INFO] Running com.example.MyServiceOrderTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceOrderTest
[INFO] Running com.example.MyServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceTest
[INFO] Running com.example.MyServiceVerifyTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceVerifyTest
[INFO] Running com.example.MyServiceVoidTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in com.example.MyServiceVoidTest
[INFO] Results:
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 0.004 s
[INFO] Finished at: 2020-06-21 10:41:55+00:00
```

Exercise 2: Parameterized Logging

Task: Write a Java application that demonstrates parameterized logging using SLF4J.

Step-by-Step Solution:

1. Add SLF4J and Logback dependencies to your `pom.xml` file:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version> </dependency>
```

2. Create a Java class that uses SLF4J for parameterized logging:

Write code for this.

```
21:13:23.350 [com.example.ParameterizedLoggingExample.main()] INFO com.example.ParameterizedLoggingExample - User name is Vineela
21:13:23.352 [com.example.ParameterizedLoggingExample.main()] INFO com.example.ParameterizedLoggingExample - User age is 22
21:13:23.352 [com.example.ParameterizedLoggingExample.main()] INFO com.example.ParameterizedLoggingExample - User salary is 50000.75
21:13:23.352 [com.example.ParameterizedLoggingExample.main()] INFO com.example.ParameterizedLoggingExample - User Vineela is 22 years old and earns 50000.75
21:13:23.352 [com.example.ParameterizedLoggingExample.main()] WARN com.example.ParameterizedLoggingExample - Low balance for user Vineela with salary 50000.75
21:13:23.352 [com.example.ParameterizedLoggingExample.main()] ERROR com.example.ParameterizedLoggingExample - Error occurred for user Vineela
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.389 s
[INFO] Finished at: 2026-06-23T21:13:23+05:30
[INFO] -----
```

Exercise 3: Using Different Appenders

Task: Write a Java application that demonstrates using different appenders with SLF4J.

Step-by-Step Solution:

1. Add SLF4J and Logback dependencies to your `pom.xml` file:

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>1.2.3</version> </dependency>
```

2. Create a `logback.xml` configuration file to define different appenders:

```

<configuration>
  <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <appender name="file" class="ch.qos.logback.core.FileAppender">
    <file>app.log</file>
    <encoder>
      <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
    </encoder>
  </appender>

  <root level="debug">
    <appender-ref ref="console" />
    <appender-ref ref="file" />
  </root>
</configuration>

```

3. Create a Java class that uses SLF4J for logging:

Write code for this.

```

[INFO]
[INFO] --- exec:3.1.0:java (default-cli) @ SLF4JExample ---
21:19:08.110 [com.example.AppenderExample.main()] DEBUG com.example.AppenderExample - This is a DEBUG message
21:19:08.111 [com.example.AppenderExample.main()] INFO com.example.AppenderExample - This is an INFO message
21:19:08.112 [com.example.AppenderExample.main()] WARN com.example.AppenderExample - This is a WARN message
21:19:08.112 [com.example.AppenderExample.main()] ERROR com.example.AppenderExample - This is an ERROR message
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 0.488 s
[INFO] Finished at: 2026-06-23T21:19:08+05:30
[INFO] -----

```